# Lab 4 – Building Microservices for your Smart Space

**B8019 - PROGRAMMING THE INTERNET OF THINGS**

**Fall 2026**

**Professor Sumi Helal**

**Due Date**: 12:00 Noon, **11 November 2026**. Submit by emailing Professor Helal and Junjie Li (lijunjie@ufl.edu). Only one member of the group submits the assignment clearly listing the full names of the group. Please include your names in the zip file name.

---

## Introduction

The Virtual Smart Space (VSS) you and your group have created in Lab1 is composed of three layers: a private cloud, one or two laptops as Edges, and a set of RPi platforms as IoT things. These layers utilize a private network (e.g., via a wireless hub) for connection, enabling the top-bottom flow of instructions (e.g., interaction commands from your IoT applications hosted in the cloud (e.g., your Metronome dashboard)) and the upstream flow of data (e.g., results of running the instructions on the RPi back to the cloud application).

In Lab 2 and 3, you and your group have focused on the first two layers of the VSS –the physical layer and the Edge layer. For the physical layer, you designed a circuit with push buttons and LEDs, and programmed your RPi platform with two modes to utilize such a circuit to implement the functionality of a metronome. While, for the Edge layer, you designed and created an HTML-based dashboard as a web interface that runs on your laptop to interact with the RPi platform remotely via RESTful APIs and MQTT/Broker.

The **services** (or functionalities) offered by IoT things like RPi platforms influence the design and implementation of IoT applications. A temperature sensor (as an example of a simple IoT thing) may offer a few services for the users, that include reporting the current temperature value in Fahrenheit or in Centigrade). While a smart lamp (as an example of a more complex IoT thing) usually offers more services, that include on/off switching, controlling the brightness and warmth, light dimming, and setting timed operations. Users and developers utilize these **available services** to compose IoT applications. The more services offered by the IoT thing, the wider the range of IoT applications that can be designed and programmed involving that IoT thing. However, to design meaningful IoT applications, users and developers must identify the **relationships** between such **services**. **Think of a relationship as a virtual link between two or more services.** While there are several relationships to consider and exploit, in this lab assignment, we will consider only the following two relationships:

**1) Order-based relationships**: Service B may run only after service A runs successfully to completion.

**2) Condition-based relationships**: Service B runs conditionally based on a specific outcome (return value) resulting from running Service A.

## Part One — Connecting multiple services to build an IoT application

In this part of Lab 4, **as a stepstone towards the final project**, you and your group will expand your work on Lab 1 and Lab 2 to implement the following:

- **Step1: Prepare an interesting and feasible application scenario**. Think of an interesting IoT application that you would like to design using the sensors and actuators available to you this semester (or that your group would personally acquire – for example a particular sensor we do not have in the class locker inventory, but that would enable your dream application idea).

- **Step2: Prepare the physical layer**. Following your idea from Step1, prepare RPi-1 platform with the sensors of choice, and RPi-2 platform (if not available use RPi-1) with the actuator of choice. You should prepare the hardware aspect (the connections) and the software functionality **(C/C++ coding – no Python allowed in this assignment**). Test this step (test that you can get sensor(s) values, and you are able to actuate the actuator(s)).

- **Step3: Prepare the Edge layer**. You will now think **services**! Your two RPi platforms must connect and report their **available services** to your Edge device. Each service is reported as a vector **(RPi identifier, service name, the number of required input parameters for the service, and the input parameter names).** For example **(*RPi-2, Read_Temp, 1, F_or_C*).** The Edge device then displays such reported **service information** for the user through a web-based dashboard.

- **Step4: Prepare a "trivial" programming environment (IDE).** The user, through the dashboard, can utilize one or both of the relationships defined above to compose a simple IoT application. **You are really designing a very small, very limited IDE at this point.** Think of a **text-based (command line)** if you will, or very simple **graphical interface (point, click, drag)** if you prefer to enter (program) such an IoT application. The former approach is obviously easier, but you are free to choose your approach.

- **Step5: Build an IoT application.** Now that you built the IDE, use it! **Compose a simple application and run it.** As you compose the application using your IDE, the Edge translates such an IoT application to a set of **service API calls**, as defined in the relationship(s), to the corresponding IoT things on the RPi platforms.

- **Step6: Display the results.** The Edge controls the order of API calls, collects the results from the IoT things, and displays the results to the user.

## Part Two — Describe and build your IoT Thing

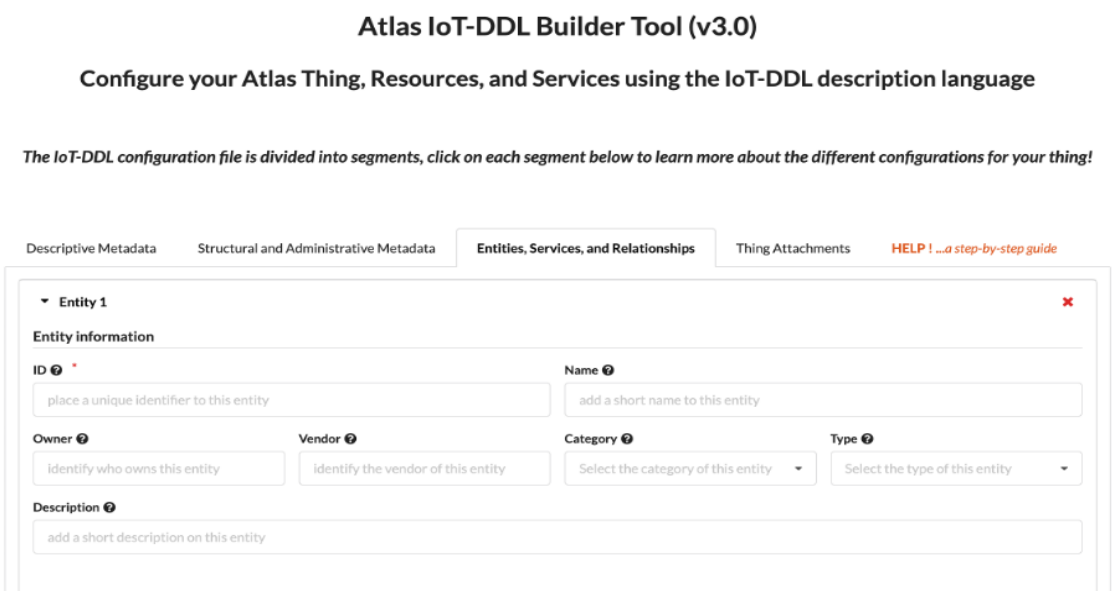
In this part of the assignment, you will use the **Atlas Thing Middleware,** which is the implementation of the key components of the Atlas Architecture. (https://github.com/NickLiJunjie/AtlasThingMiddleware_RPI).

Atlas thing middleware (covered in class lectures) is an operating layer (firmware) embedded within an IoT thing that allows for the automatic integration of the IoT thing with the rest of the VSS smart space (integrating with other IoT things). The middleware enables direct communication between IoT things, and between IoT things and users and the cloud. The middleware and its **IoT Device Description Language (IoT-DDL)** allow the users (e.g., device owners) **to define and use the services provided by these IoT things**. **They also allow IoT programmers to discover and combine available services to create IoT Applications in a smart space**.

**Step1:**

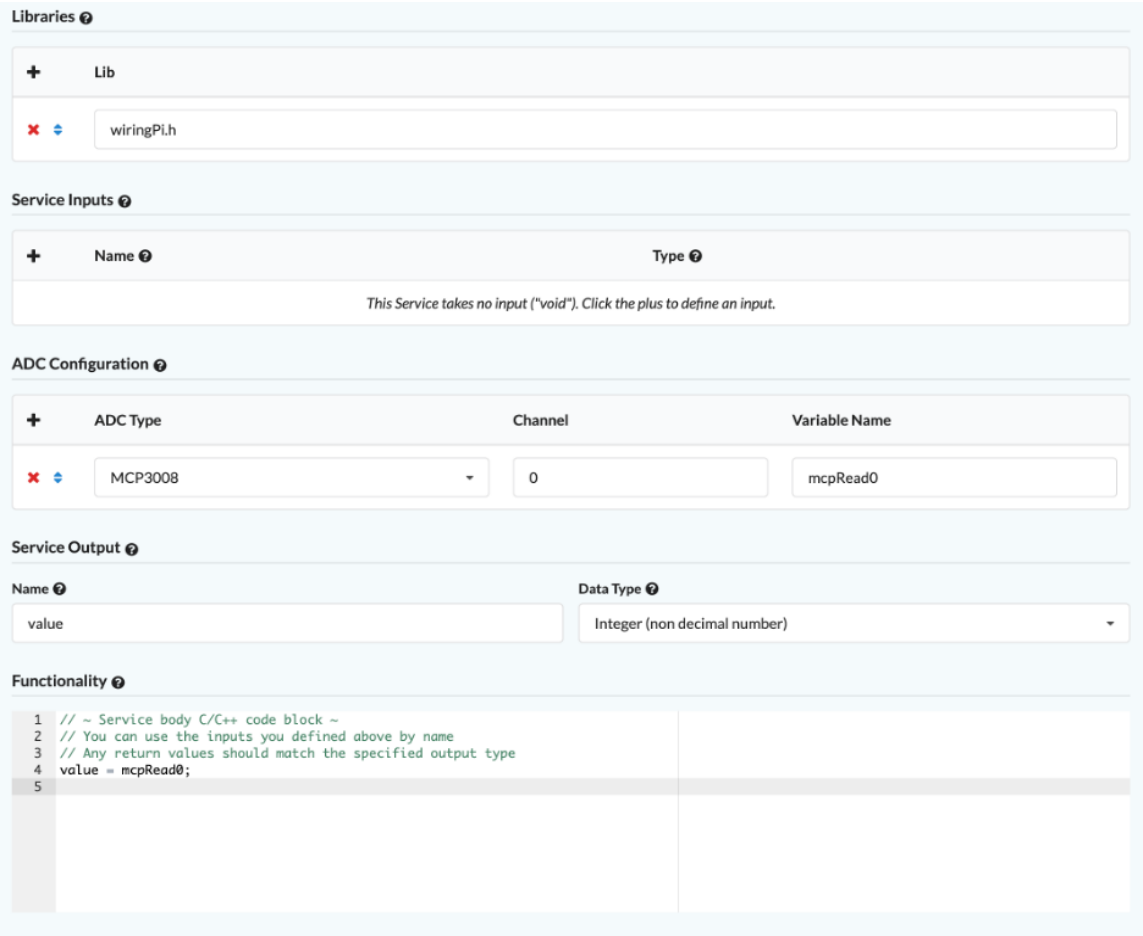
The Atlas thing middleware requires the use of the IoT-DDL device description language to configure an Atlas thing and enable it to perform a wide set of functionalities (e.g., advertisement and discovery, dynamically creating microservices, and enabling users and other IoT things to use and invoke such services through their respective APIs).

IoT-DDL (covered or will soon be covered in class lectures) is an XML-based declarative language that describes a thing in terms of inner components, identity, capabilities, resources, and services (in addition to other elements to be covered later). In this step, you will use the **IoT-DDL Builder web-based tool** (https://www.cise.ufl.edu/~helal/Atlas-IoT-DDL-Builder.html) to build an IoT-DDL specification file that describes your things you developed in Part One.



Consider each of the two Raspberry Pi's (or the only single Pi you may have available) as a thing with one or more inner entities (called bit-things). Under "*Descriptive Metadata*", you can describe the main attributes and parameters of a thing. Under "*Entities, Services, and Relationships*" you can create inner entities **and include the C code of the services you designed in Part One** provided by each entity. Create a separate entity for each bit thing (e.g., entity for each sensor and entity for each actuator).

For each service you create, if your service involves GPIO, you need to add the required library header files in "*Libraries*" first. You can also specify the expected inputs for the service (no inputs by default) under "*Service Inputs*". Under "*ADC Configuration*" (no ADC by default), you can select the ADC model and specify the Channel, and store the value read by the ADC in a variable with the variable name you specify, as shown in the figure below. Under "*Service Output*", you can specify the expected output (the service returns void by default). Under " *Functionality* ", you should develop the C/C++ code for this service (you should import and "reuse" the code you developed in the first part). For example, you develop a service - as shown below - that does not accept inputs, but simply reads the value of the ADC and returns the output value (named value). In the function window, you can start developing the C/C++ code and use "mcpRead0" and "value" in the code (no need to include the return statement, the middleware will automatically include this statement).



At this point, you would have created your first IoT-DDL specification for all bit-things that you will attach to your Raspberry Pi things.

An example IoT-DDL for a Servo motor entity (attached to a Raspberry Pi) offering a single service can be viewed using this link:

https://github.com/NickLiJunjie/AtlasFramework/blob/main/IoT-DDL/Servo_Motor.xml

**Step2:**

In this step you will **import the Atlas thing middleware to your Raspberry Pi** and **add the IoT-DDL file you developed** in Step1 by following the instructions as detailed in this link:

https://github.com/NickLiJunjie/AtlasThingMiddleware_RPI

It is important that you make sure the IoT-DDL filer is indeed placed in the correct directory inside the middleware, as per the instructions given via the link above.

**Step3:**

When you run the Atlas middleware at end of Step2, it will parse the different parts of the IoT-DDL you included, generate microservices from the services' descriptions, build JSON-based tweet messages summarizing the parameters of the IoT-DDL, and start broadcasting these tweets to the VSS smart space. Other things, or application software deployed in the same smart space can capture these tweets and take advantage of them in anyway meaningful.

In this step, you will create a demo app named "Demo IoT Application" in the VSS that simply invokes the services you described in the IoT-DDL through their appropriate API calls. Design and implement this Demo IoT Application using the programming language you find appropriate (e.g., C/C++, Java, Python) to unicast API calls (to the IP of your thing over port address 6668). The API call is a JSON-based message, structured as follows:

```
{ "Tweet Type"   : Service call,
 "Thing ID"    : the id of your smart thing as declared in the IoT-DDL,
 "Space ID"    : the id of your smart space as declared in the IoT-DDL,
 "Service Name"  : the name of the function you would like to call,
 "Service Inputs" : (list of expected inputs)
};
```

Sending such call to an Atlas thing triggers the appropriate handles and the thing responds with JSON-based results back.

Let us assume, in the IoT-DDL, you made *MySmartThing01* as your smart thing id, and *MySmartSpace* as your smart space id. You also developed two services, *Service1* that accepts no inputs and *Service2* that accepts two integer inputs. The structure of the API calls in this example will therefore be:

```
char Call1[] = "

{
"Tweet Type": "Service call",
  "Thing ID": "MySmartThing01",
  "Space ID": "MySmartSpace",
  "Service Name": "Service1",
  "Service Inputs": "()"
}";
```

and,

```
char Call2[] = "

{
"Tweet Type": "Service call",
  "Thing ID": "MySmartThing01",
  "Space ID": "MySmartSpace",
  "Service Name": "Service2",
  "Service Inputs": "(3,9)"
}";
```

You should comment your code adequately to explain what specific demonstration it accomplishes.

## Extra-Credit: Virtual Smart Space Across Two Groups (10% extra credit)

You will be able to call and test your group thing services from one of your development machines as long as the machine is on the same network (direct Ethernet connection or WiFi other than UNIBO's network, as described in the previous lab document).

For extra credit, two groups may attempt to create a cross-group "virtual smart space" (VSS) via an external VPN server that will allow the thing services belonging to the two groups to interact with each other (tweet and hear the tweets across the two groups, and of course invoking services across). You may use an OpenVPN server and host it somewhere. If you opt for Open VPN, you should install it on your RPi with **sudo apt install openvpn**, and should create an **OpenVPN configuration file** and place it in the **/etc/openvpn/client** directory in your RPi's, renaming it to **vss.conf**. To connect to the VSS, run the command **sudo systemctl start openvpn-client@vss**. After a second, you should see the VPN connection running successfully with **sudo systemctl status openvpn-client@vss**. If there was an error, ensure the configuration file has been named correctly and is in the right location.

You will now have an IP address within the cross-group VSS, likely in the 10.254.0.0/24 range. If you run ip -br a, you should see the IP under the tap0 interface. Your device and its

services are now visible and available to the other group's devices and services. To disconnect from the VSS, run **sudo systemctl stop openvpn-client@vss**.

**NOTE:** Please follow good IoT security practices and change the password (with passwd) for your RPi to something from the default, since it is now visible to others outside your group.

## Submission Details

You will submit a *.zip* file containing a folder for each part (folder names: PartOne and PartTwo). For Part One, you should include the C/C++ service implementations (one or more services for each one of your RPi platforms) as a **main(){}** program with the services as functions. For Part Two, use a separate folder to include the IoT-DDL file you have generated using the Builder tool. Also include the code for the demo IoT application you have developed to use the services. Remember, comments (documentations) are very important and do affect your final grades as per the rubric. Keep track of who did what in the group and include a readme file in the folder for Part One, that includes a short description of the distribution of effort. Be ready to demo and go over your code with the TAs during lab grading. All group members must be available and present during the online grading sessions. Absences will earn their absentees a grade deduction.

# Appendix

## Setting up ADC connection

### *ADC Introduction*

Currently Atlas Framework supports two ADCs, MCP3008 and ADS1115. The MCP3008 is an 8-channel 10-bit ADC that communicates using the SPI protocol and the ADS1115 is a more advanced 4-channel 16-bit ADC that communicates via I2C.

### *MCP3008*

#### Enable SPI

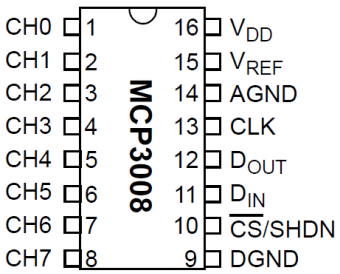
SPI (Serial Peripheral Interface) is necessary for communicating with devices like the MCP3008.

- Open the terminal and type:

```
sudo raspi-config
```

- Navigate to "Interface Options"
- Select "SPI"
- Choose "Yes" to enable SPI

#### Wiring up

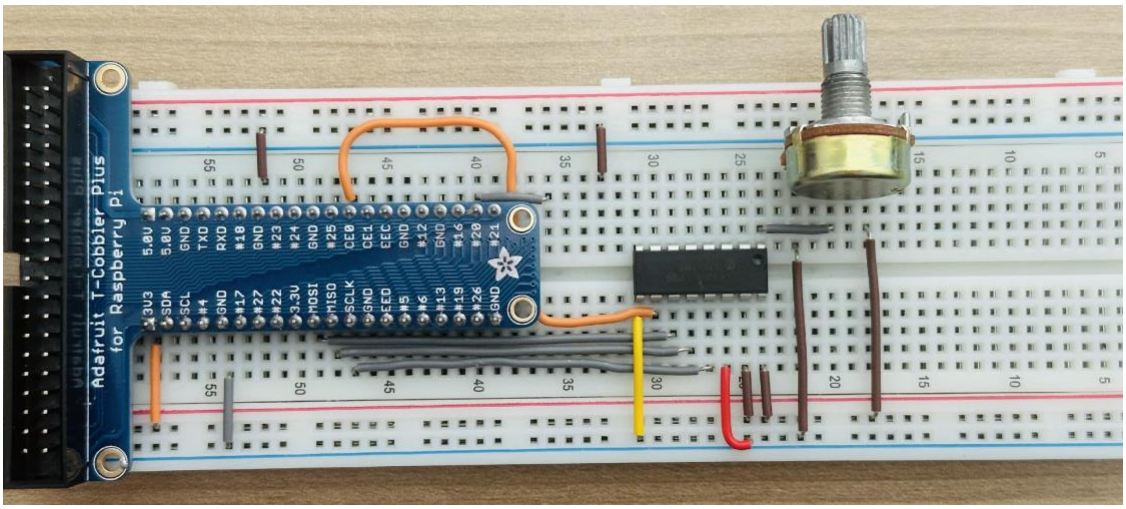



| MCP3008 Pin | Raspberry Pi Pin | Description |
|---|---|---|
| VDD | 3.3V | Power supply for the MCP3008 |
| VREF | 3.3V | Reference voltage for analog conversion |
| AGND | GND | Analog ground connection |
| DGND | GND | Digital ground connection |
| CLK | GPIO18 (Pin 12) | SPI Clock signal |
| DOUT | GPIO23 (Pin 16) | Data Out from MCP3008 to Raspberry Pi (MISO) |

| MCP3008 Pin | Raspberry Pi Pin | Description |
|---|---|---|
| DIN | GPIO24 (Pin 18) | Data In from Raspberry Pi to MCP3008 (MOSI) |
| CS/SHDN | GPIO25 (Pin 22) | Chip Select/Shutdown pin |
| CH0-CH7 | Analog sensors | Analog input channels to connect your sensors |

**Notes:**

- This is a software SPI configuration
- You can swap the CLK, DOUT, DIN, and CS/SHDN pins to any other free GPIO pins on the Raspberry Pi
- If you change the pins, you'll need to modify your code accordingly
- The MCP3008 has 8 analog input channels (CH0-CH7) compared to ADS1115's 4 channels




*ADS1115*

## Enable I2C

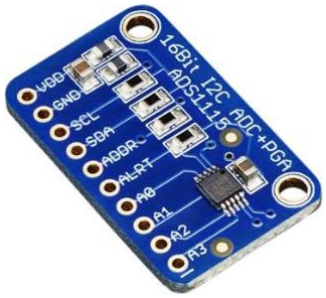
I2C is essential for communicating with devices like the ADS1115.

- Open the terminal and type:

```
sudo raspi-config
```

- Navigate to "Interface Options"
- Select "I2C"
- Choose "Yes" to enable I2C

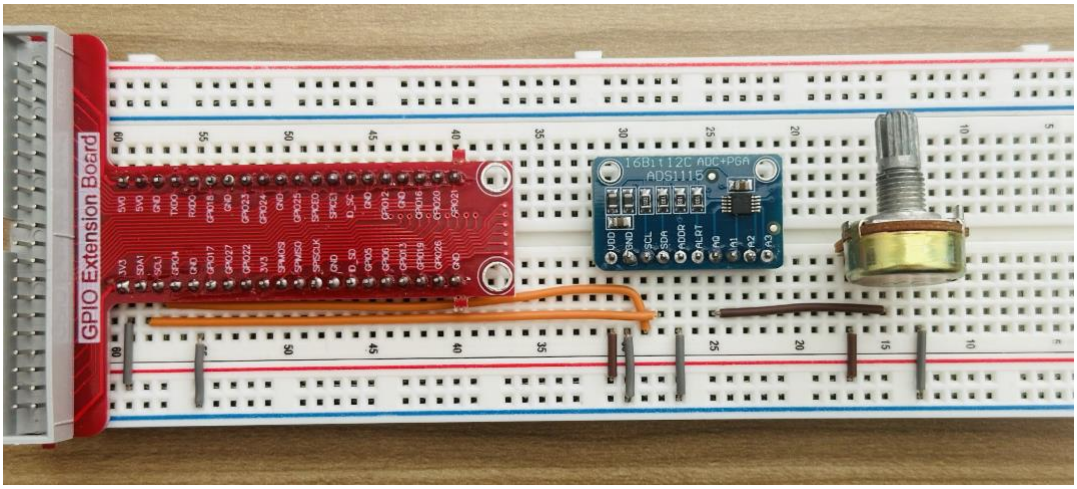
## Wiring up




| ADS1115 Pin | Raspberry Pi Pin | Description |
|---|---|---|
| VCC | 3.3V | Power supply for the ADS1115 (Do not use 5V as it may damage the chip) |
| GND | GND | Ground connection |
| SDA | GPIO2 (Pin 3) | I2C Serial Data line for communication |
| SCL | GPIO3 (Pin 5) | I2C Serial Clock line for synchronizing data transfer |
| ADDR | GND (optional) | Address select pin (connect to GND for default address 0x48) |
| ALRT | Not connected | Alert/Ready pin (optional, can be used for conversion-ready signaling) |
| A0-A3 | Analog sensors | Analog input channels to connect your sensors |

**Notes:**

- The Raspberry Pi's I2C pins (SDA/SCL) must be enabled via raspi-config as explained previously
- The ADS1115 operates on 3.3V - never connect it to 5V
- You can connect up to 4 analog sensors to A0-A3 pins
- The default I2C address is 0x48 when ADDR is connected to GND




## Using ADC in Atlas Framework

After setting up your Raspberry Pi and connecting the hardware, you need to specify the appropriate library in IoT-DDL to use ADC functionality. Currently, only wiringPi is supported, so when using ADC, you must manually add "wiringPi.h" to the "Libraries" section.

### *Resolution Differences*

The resolution of your ADC will affect the output values:

- **MCP3008**: With its 10-bit resolution, the Atlas Framework will output integers between 0 and 1024 (2^10) when reading from this ADC.
- **ADS1115**: With its 16-bit resolution, the Atlas Framework will output integers between 0 and 65536 (2^16) when reading from this ADC.

### *Sensor Value Conversion*

If you have connected sensors to your ADC, you'll need to calculate the actual sensor values from the ADC output integers. This usually involves applying a conversion formula specific to your sensor type, which transforms the raw ADC readings into meaningful measurements like temperature, light intensity, or voltage.

Remember to add the appropriate scaling factors and offsets in your code to convert the raw ADC readings to your desired units of measurement.